



AtliQ Hardware:
Sales & Analytics

REPORTING AUTOMATION(*Store Procedure*)



Business Requirement

- **Business Requirement-** The Product Owner requested a monthly product-level sales report for Croma India customer for FY 2021.
- **Obstacle-** Manual data aggregation processes were significantly hindered by complex Fiscal Year pricing logic.

Technical Execution

- Created a custom SQL function `get_fiscal_year(date)` to automatically convert calendar dates into corresponding fiscal years for accurate financial reporting.
- After creating `get_fiscal_year(date)`, created a store procedure to get the individual product wise sales (aggregated on monthly basis).

Croma India product wise sales report for fiscal year 2021

Attach Add a child issue Link issue

Description

Normal text B I ... A ...

As a product owner, I want to generate a report of individual product sales (aggregated on a monthly basis at the product code level) for Croma India customer for FY=2021 so that I can track individual product sales and run further product analytics on it in excel.

The report should have the following fields,

1. Month
2. Product Name
3. Variant
4. Sold Quantity
5. Gross Price Per Item
6. Gross Price Total

Requirement

```
1 • CREATE DEFINER='root'@'localhost' FUNCTION `get_fiscal_year`(  
2     calendar_date date  
3 ) RETURNS int  
4     DETERMINISTIC  
5 BEGIN  
6  
7     declare fiscal_year INT;  
8     SET fiscal_year=year(date_add(calendar_date, INTERVAL 4 MONTH));  
9  
10    RETURN fiscal_year;  
11 END
```

Function-
get_fiscal_year()

```
1 • SELECT  
2     s.date, p.product, p.variant, s.sold_quantity, g.gross_price,  
3     g.gross_price*s.sold_quantity AS total_gross_price  
4 FROM fact_sales_monthly s  
5 JOIN dim_product p  
6 ON s.product_code=p.product_code  
7 JOIN fact_gross_price g  
8 ON s.product_code=g.product_code AND g.fiscal_year=get_fiscal_year(s.date)  
9 WHERE customer_code="90002002" AND get_fiscal_year(date)=2021  
10 ORDER BY s.date ASC  
11 LIMIT 10000000
```

Store
Procedure

Call stored procedure gdb0041.get_monthly_gross_sales_f...

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

customer_code [IN] TEXT

Execute Cancel

date	product	variant	sold_quantity	gross_price	total_gross_price
2020-09-01	AQ Dracula HDD - 3.5 Inch SATA 6 Gb/s 5400 RPM 256 MB Cache	Premium Plus	146	22.9729	3354.0434
2020-09-01	AQ Dracula HDD - 3.5 Inch SATA 6 Gb/s 5400 RPM 256 MB Cache	Standard	149	23.6987	3531.1063
2020-09-01	AQ Dracula HDD - 3.5 Inch SATA 6 Gb/s 5400 RPM 256 MB Cache	Plus	107	24.7312	2646.2384
2020-09-01	AQ Dracula HDD - 3.5 Inch SATA 6 Gb/s 5400 RPM 256 MB Cache	Premium	123	23.6154	2904.6942
2020-09-01	AQ WereWolf NAS Internal Hard Drive HDD - 8.89 cm	Standard	146	23.7223	3463.4558
2020-09-01	AQ Zion Saga	Plus	236	27.1027	6396.2372
2020-09-01	AQ Zion Saga	Resplendent	137	28.0050	3836.6850



Problem & Logic

- **Problem-** The marketing team faced delays in identifying high-growth regions due to manual tracking of sales data across multiple fiscal periods.
- **Logic-** If **Total Sold Quantity > 5 Million**, then the market is labeled is **“Gold”**, else **“Silver”**.

It is allowing the marketing team to quickly identify regions with strong sales performance.

Technical Execution

- The system compares the aggregated **SUM(sold_quantity)** against a predefined threshold to automatically classify performance as **“Gold”** or **“Silver”**.

Stored proc for market badge

Attach

Add a child issue

Link issue



Description

Create a stored proc that can determine the market badge based on the following logic.

If **total sold quantity > 5 million** that market is considered **Gold** else it is **Silver**

My input will be,

- market
- fiscal year

Output

- market badge

Requirement

```
1 • CREATE DEFINER='root'@'localhost' PROCEDURE `get_market_badge`(  
2   IN market VARCHAR(45),  
3   IN fiscal_year INT,  
4   OUT market_badge VARCHAR(45)  
5 )  
6 BEGIN  
7   declare qty INT default 0;  
8   #default value is india  
9   IF market="" THEN SET market="India"; end if;  
10  # to_retrieve_the_sold_quantity  
11  SELECT  
12    sum(s.sold_quantity) INTO qty  
13  FROM fact_sales_monthly s  
14  JOIN dim_customer c  
15    ON s.customer_code=c.customer_code  
16  WHERE c.market=market AND get_fiscal_year(s.date)=fiscal_year  
17  GROUP BY c.market;  
18  # market_badge_condition  
19  IF qty>5000000 THEN SET market_badge= "GOLD";  
20  ELSE SET market_badge= "SILVER";  
21  end if;  
22  END
```

Call stored procedure gdb0041.get_market_badge

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

market		[IN]	VARCHAR(45)
fiscal_year		[IN]	INT
market_badge		[OUT]	VARCHAR(45)

Execute Cancel

Store Procedure

```
1 • set @market_badge = '0';  
2 • call gdb0041.get_market_badge('india', 2021, @market_badge);  
3 • select @market_badge;  
4
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	@market_badge			
	GOLD			

Final Result



Objective

The goal was to track monthly aggregated **gross sales** for major customers like “**Croma**” in order to evaluate account health and strengthen business relationships.

Technical Execution

Developed a SQL stored procedure to automate the generation of monthly gross sales reports for key accounts such as Croma.

Gross monthly total sales report for Croma

Attach

Add a child issue

Link issue

▼

...

Description

As a product owner, I need an aggregate monthly gross sales report for Croma India customer so that I can track how much sales this particular customer is generating for AtliQ and manage our relationships accordingly.

The report should have the following fields,

1. Month
2. Total gross sales amount to Croma India in this month

Requirement

```
1 • CREATE DEFINER='root'@'localhost' PROCEDURE `get_monthly_gross_sales_for_customer`(  
2     customer_code TEXT  
3 )  
4 BEGIN  
5     SELECT  
6         s.date,  
7         SUM(g.gross_price*s.sold_quantity) as total_gross_price  
8     FROM fact_sales_monthly s  
9     JOIN fact_gross_price g  
10    ON s.product_code=g.product_code AND  
11       g.fiscal_year=get_fiscal_year(s.date)  
12    WHERE  
13       find_in_set(s.customer_code, customer_code)>0  
14    GROUP BY s.date;  
15 END
```

Call stored procedure gdb0041.get_monthly_gross_sales_f...

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

customer_code [IN] TEXT

Execute Cancel

Store Procedure

```
1 • call gdb0041.get_monthly_gross_sales_for_customer('90002002');
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
date	total_gross_price			
2017-09-01	122407.5582			
2017-10-01	162687.5716			
2017-12-01	245673.8042			
2018-01-01	127574.7372			
2018-02-01	144799.5182			
2018-04-01	130643.8976			
2018-05-01	139165.0975			

Final Result



Requirement & Logic

- A consolidated view of the top-performing markets, products, and customers ranked by net sales for a selected financial year.
- **Objective-** Detect strong sales trends and potential market risks to improve budget planning and operational efficiency.
- **Final Solution-** Final Solution: Created flexible SQL stored procedures with parameter-driven ranking logic for customizable Top N analysis.

Technical Execution

Developed dynamic **stored procedures** such as ***get_top_n_market_net_sales*** with parameterized inputs for Fiscal Year and Top N ranking limits.

Description

As a product owner, I want a report for top markets, products, customers by net sales for a given financial year so that I can have a holistic view of our financial performance and can take appropriate actions to address any potential issues.

We will probably write stored proc for this as we will need this report going forward as well.

1. Report for **top markets**.

Rank	Market	Net Sales (in millions)
1	India	210.67
2	USA	132.05
3	South Korea	64.01

2. Report for **top products**.

Rank	Product	Net Sales
1	AQ BZ Allin1	33.75
2	AQ Qwerty	27.84

Requirement

```
1 • CREATE DEFINER='root'@'localhost' PROCEDURE `get_top_n_markets_by_net_sales`(  
2     in_fiscal_year INT,  
3     in_top_n int  
4 )  
5 BEGIN  
6     SELECT  
7     market,  
8     ROUND(SUM(net_sales)/1000000,2) as net_sales_mLn  
9     FROM net_sales  
10    where fiscal_year=in_fiscal_year  
11    group by market  
12    ORDER BY net_sales_mLn DESC  
13    LIMIT in_top_n;  
14 END
```

Call stored procedure gdb0041.get_top_n_markets_by_net... — □ ×

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

in_fiscal_year [IN] INT

in_top_n [IN] int

Execute Cancel

Store
Procedure

```
1 • call gdb0041.get_top_n_markets_by_net_sales(2020, 5);
```

market	net_sales_mLn
India	64.73
USA	46.35
South Korea	22.38
Philippines	17.45
Canada	15.87

Final
Result